



Programmation fonctionnelle avec Scala

Yidhir MOUDOU B
yidhir.moudoub@outlook.com
Cours IABD

2026

Présentation de l'auteur

- ▶ **Nom** : Yidhir MOUDOUB
- ▶ **Poste** : Responsable de l'équipe Data Engineering @TF1
- ▶ **Contact** : yidhir.moudoub@outlook.com
- ▶ **Domaines d'expertise** :
 - Écosystème Hadoop (HDFS, MapReduce, Spark)
 - Programmation fonctionnelle avec Scala
 - Spark Core et Spark SQL
 - Spark Streaming et Structured Streaming
 - Traitement de données distribuées

Sommaire

1. Prérequis
2. Pourquoi la programmation fonctionnelle ?
3. Session 1 — Bases de Scala
Exercices : Session 1
4. Session 2 — Collections
Exercices : Session 2
5. Session 3 — Pipelines fonctionnels
Exercices : Session 3
6. Session 4 — Programmation orientée objet

- Exercices : Session 4
7. Session 5 — Pattern matching
Exercices : Session 5
8. Session 6 — Génériques et traits
Exercices : Session 6
9. Session 7 — Concurrency avec les Futures
Exercices : Session 7
10. Résumé
11. Trimestre suivant : projet Scala & Spark

► SECTION 1

Prérequis

Prérequis

Connaissances requises

- ▶ Connaissance d'au moins un langage de programmation de haut niveau.
- ▶ Familiarité avec les structures de données de base (listes, maps, sets).
- ▶ Expérience préalable des concepts de programmation orientée objet.

Un plus

Une expérience avec Java ou un autre langage JVM est un plus, mais n'est pas requise.

► SECTION 2

Pourquoi la programmation fonctionnelle ?

Impératif vs fonctionnel

Impératif

- ▶ Décrit **comment** le faire.
- ▶ État mutable, boucles, effets de bord.
- ▶ Plus difficile à raisonner en contexte concurrent.

Fonctionnel

- ▶ Décrit **ce qu'il faut** calculer.
- ▶ Valeurs immuables, expressions, fonctions pures.
- ▶ Composable, testable, adapté au parallélisme.

Pourquoi Scala ?

Points forts

- ▶ **Hybride** : combine POO et programmation fonctionnelle dans un seul langage.
- ▶ **JVM** : interopérabilité complète avec les bibliothèques Java.
- ▶ **Système de types fort** : détecte les erreurs à la compilation.
- ▶ **Concis** : syntaxe expressive sans boilerplate.
- ▶ **Industrie** : propulse Apache Spark, Akka, Play Framework.

► SECTION 3

Session 1 — Bases de Scala

Variables, types et I/O

`val` vs `var`

- ▶ `val name = "Alice"` — immuable (préfér  en FP).
- ▶ `var count = 0` — mutable (  utiliser avec parcimonie).
- ▶ Les types sont inf r s ; annotation explicite : `val x: Int = 42`.

Interpolation de cha nes et I/O

- ▶ `s"Hello $name"` — interpolation s.
- ▶ `println(...)` — sortie standard.
- ▶ `scala.io.StdIn.readLine()` — lit une saisie utilisateur.

Structures de contrôle

if comme expression

En Scala, `if/else` renvoie une valeur :

```
val abs = if (x > 0) x else -x
```

Boucles for et intervalles

- ▶ `for (i ← 1 to 10) println(i)`
- ▶ `for (i ← 1 until 10)` — exclut la borne supérieure.
- ▶ Les compréhensions `for` renvoient des collections avec `yield`.

Exercices Session 1 : bases de Scala

- 1 **Affichage simple** : déclarer une variable name, afficher Hello <name> avec interpolation de chaîne, puis afficher les 10 premiers entiers avec une boucle for.
- 2 **Jeu du nombre mystère** : tirer un nombre aléatoire entre 0 et 100, demander à l'utilisateur de deviner, indiquer trop haut/trop bas après chaque tentative, afficher le nombre d'essais.

Solutions

→ [Exercice 1](#) → [Exercice 2](#)

▶ SECTION 4

Session 2 — Collections

Collections principales

List

- ▶ Liste chaînée immuable. **Ajout en tête** rapide (`::`), accès aléatoire lent.
- ▶ `val nums = List(1, 2, 3)`

Vector

- ▶ Indexé, immuable. **Accès aléatoire** et **ajout en fin** rapides.
- ▶ `val v = Vector(1, 2, 3)`

Map et Set

- ▶ `Map[K, V]` — table clé-valeur. Créée avec `Map("a" → 1)`.
- ▶ `Set[T]` — éléments uniques, test d'appartenance rapide.

Performance List vs Vector

Opération	List	Vector
Ajout en tête ($x :: xs$)	$O(1)$	$O(\log n)$
Ajout en fin ($xs :+ x$)	$O(n)$	$O(\log n)$
Accès aléatoire	$O(n)$	$O(\log n)$
Tête	$O(1)$	$O(\log n)$

Règle générale

Utilisez List pour les algorithmes récursifs (head/tail). Utilisez Vector pour un ajout rapide ou un accès indexé.

Exercices Session 2 : collections

- 1 **Comptage de mots** : générer `List("A", "B", "A", "C", "B", "A")`, calculer les occurrences de chaque mot avec `groupBy`, stocker le résultat dans un `Map[String, Int]`.
- 2 **List vs Vector** : créer une collection de 100 000 éléments. Mesurer et comparer l'ajout en tête (`0 :: col`) vs l'ajout en fin (`col :+ 0`) pour les deux types.

Solutions

→ [Solutions de la Session 2 sur GitHub](#)

► SECTION 5

Session 3 — Pipelines fonctionnels

Fonctions d'ordre supérieur

Le trio fondamental

- ▶ **filter(p)** : conserve les éléments correspondant au prédicat p.
- ▶ **map(f)** : transforme chaque élément avec la fonction f.
- ▶ **reduce(f) / fold** : agrège tous les éléments.

Chaînage

Combinez-les en un pipeline déclaratif :

```
list.filter(_ % 2 == 0).map(_ * 3).sum
```

groupBy et case class

groupBy

Regroupe les éléments par une fonction clé et renvoie un Map :

```
words.groupBy(identity).mapValues(_.length)
```

case class

Conteneur de données immuable avec equals, hashCode, copy et pattern matching gratuits :

```
case class Record(name: String, value: Int)
```

Exercices Session 3 : pipelines fonctionnels

- 1 **processNumbers** : prendre une `List[Int]`, conserver les nombres pairs, multiplier par 3, renvoyer la somme.
Exemple : `List(1,2,3,4,5,6)` $\rightarrow$ 36
- 2 **Pipeline de texte** : découper une chaîne en mots, mettre en minuscules, conserver les mots de longueur ≥ 4 , compter les occurrences.

Solutions

→ [Solutions de la Session 3 sur GitHub](#)

► SECTION 6

Session 4 — Programmation orientée objet

Classes et objets

Définition de classe

- ▶ Constructeur principal dans l'en-tête de la classe : `class Foo(val x: Int)`.
- ▶ Méthodes définies avec `def`.
- ▶ `object` compagnon pour les membres statiques et les factory methods.

Immuabilité avant tout

Préférez les champs `val` et renvoyez de nouvelles instances plutôt que de muter l'état.

Héritage et polymorphisme

Étendre des classes

- ▶ `class Car extends Vehicle` — héritage simple.
- ▶ `override def move(): Unit = ...` — override explicite requis.

Polymorphisme

Stockez des instances de sous-classes dans une `List[Vehicle]` et appelez `move()` sur chacune — la bonne implémentation est choisie à l'exécution.

Exercices Session 4 : POO

- 1 **Classe Rectangle** : attributs `width` et `height`; méthodes `area` et `perimeter`. Instancier et afficher les deux valeurs.
- 2 **Hiérarchie de classes** : classe de base `Vehicle` avec `move()`; sous-classes `Car` et `Bike` qui la surchargent. Stocker les véhicules dans une liste et appeler `move()` de façon polymorphe.

Solutions

→ [Solutions de la Session 4 sur GitHub](#)

▶ SECTION 7

Session 5 — Pattern matching

Pattern matching

i L'expression match

- ▶ Remplace les chaînes `switch` et `instanceof`.
- ▶ Filtre par **valeur**, **type**, **structure** et **gardes**.
- ▶ L'exhaustivité est vérifiée par le compilateur pour les hiérarchies `sealed`.

i Patterns courants

- ▶ **Littéral** : `case 0 ⇒`
- ▶ **Type** : `case s: String ⇒`
- ▶ **Garde** : `case n: Int if n > 0 ⇒`
- ▶ **Structurel** : `case List(h, _*) ⇒`

Gestion d'erreurs avec Either

Either[L, R]

- ▶ Left(error) — représente un échec.
- ▶ Right(value) — représente un succès.
- ▶ Entièrement composable : flatMap, map, fold.

Préférez Either aux exceptions

Rend les chemins d'erreur **explicites dans la signature de type** — l'appelant est forcé de gérer les deux cas.

Exercices Session 5 : pattern matching

- 1 **describe(x: Any)** : utiliser le pattern matching pour afficher :
« *Positive number* » ($\text{Int} > 0$), « *Empty string* », « *List of length N* », « *Unknown type* ».
- 2 **safeSqrt** : renvoyer `Either[String, Double]` — `Left("Negative number")` si $x < 0$, sinon `Right(sqrt(x))`.

Solutions

→ [Solutions de la Session 5 sur GitHub](#)

► SECTION 8

Session 6 — Génériques et traits

Génériques

i Paramètres de type

- ▶ `def firstElement[T](list: List[T]): Option[T]`
- ▶ Écrit une fois, utilisable pour *n'importe quel* type — type-safe sans cast.
- ▶ Bornes : `[T <: Shape]` (supérieure), `[T >: Shape]` (inférieure).

i Option[T]

- ▶ `Some(value)` ou `None` — plus de null pointer exceptions.
- ▶ Se compose avec `map`, `flatMap`, `getOrElse`.

Traits et hiérarchies scellées

Traits

- ▶ Interface + implémentation optionnelle.
- ▶ Une classe peut combiner plusieurs traits : `class Foo extends A with B.`

Trait scellé (`sealed`)

- ▶ Toutes les sous-classes doivent être dans le même fichier.
- ▶ Permet un pattern matching **exhaustif** — le compilateur avertit en cas de cas manquant.

Types de données algébriques

`sealed trait Shape` avec `case class Circle` et `case class Rectangle` est le pattern ADT idiomatique en Scala.

Exercices Session 6 : génériques & traits

- 1 **firstElement[T]** : fonction générique renvoyant `Option[T]` — `Some(head)` pour les listes non vides, `None` sinon.
- 2 **Shape abstrait** : `sealed trait Shape` avec une méthode `area` ; implémenter `Circle(radius)` et `Rectangle(w, h)`.

Solution

→ [Solution de la Session 6 sur GitHub](#)

► SECTION 9

Session 7 — Concurrency avec les Futures

Calcul asynchrone

`Future[T]`

- ▶ Représente une valeur qui sera disponible **éventuellement**.
- ▶ Nécessite un `ExecutionContext` implicite (pool de threads).
- ▶ Non-bloquant par défaut; se compose avec `map`, `flatMap`, `recover`.

`Await.result`

Ne bloquez le thread que quand vous *devez* attendre un résultat (ex. fin du programme). À éviter dans les chemins critiques en production.

Pattern de calcul parallèle

Exemple : somme parallèle

- 1 Diviser un grand Vector en deux moitiés avec `splitAt`.
- 2 Lancer deux Futures qui somment chacune leur moitié en parallèle.
- 3 Combiner les résultats avec une compréhension `for` ou `zip`.

Règle clé

Créez les Futures **avant** de les combiner — les imbriquer dans une compréhension `for` les rend séquentielles.

Exercices Session 7 : concurrence

- 1 **Future basique** : créer une Future qui calcule une valeur de façon asynchrone, utiliser `Await.result` pour la récupérer et l'afficher.
- 2 **Somme parallèle** : diviser un Vector en deux moitiés, calculer chaque somme dans une Future séparée en parallèle, combiner et afficher le total.

Solutions

→ [Exercice 1](#) → [Exercice 2](#)

► SECTION 10

Résumé

Vue d'ensemble du cursus

Session	Sujet	Concept clé
1	Bases de Scala	<code>val</code> , interpolation, boucles
2	Collections	<code>List</code> , <code>Vector</code> , <code>Map</code>
3	Pipelines FP	<code>filter</code> / <code>map</code> / <code>fold</code> , <code>groupBy</code>
4	POO	Classes, traits, héritage
5	Pattern Matching	<code>match</code> , <code>Either</code>
6	Génériques	<code>[T]</code> , traits scellés, ADT
7	Concurrence	<code>Future</code> , <code>ExecutionContext</code>

Points clés à retenir

i Principes fonctionnels à retenir

- 1 Privilégiez l'**immuabilité** : `val` plutôt que `var`.
- 2 Utilisez des **expressions**, pas des instructions — tout renvoie une valeur.
- 3 Composez de petites **fonctions pures** en pipelines plus larges.
- 4 Utilisez les **types** pour rendre les états illégaux irreprésentables (`Option`, `Either`, traits scellés).
- 5 Le parallélisme est facile quand il n'y a **pas d'état mutable partagé**.

Ressources

Dépôt des exercices

→ github.com/Yidhir-sudo/Scala-course

Pour aller plus loin

- ▶ *Programming in Scala* — Odersky, Spoon, Venners.
- ▶ *Functional Programming in Scala* — Chiusano & Bjarnason.
- ▶ Documentation officielle : <https://docs.scala-lang.org>

► SECTION 11

Trimestre suivant : projet Scala & Spark

Vue d'ensemble du projet

Objectif

Construire un **pipeline de prédiction ML** avec trois microservices Scala, un script d'entraînement Python, et un dashboard de visualisation Streamlit.

Stack technique

- ▶ **Scala** – trois microservices (parsing, scoring, prédiction)
- ▶ **Apache Spark** – traitement de données distribué dans chaque service
- ▶ **Python** – entraînement du modèle ML (scikit-learn / MLlib)
- ▶ **Streamlit** – visualisation des prédictions par lots

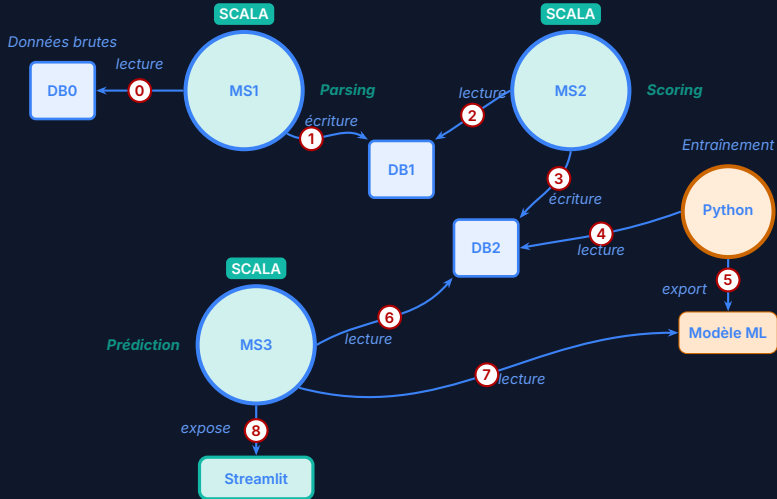
Organisation

- ▶ Groupes de **3 à 5 étudiants**
- ▶ Tout le code versionné sur **GitHub**
- ▶ Les services communiquent via une **couche de stockage partagée** (Parquet / BDD)

Contrainte clé

Aucun appel direct entre services — chaque échange passe par le stockage partagé.

Architecture du projet



Les trois microservices

i MS1 – Parsing

- ▶ Lit les données brutes d'entrée
- ▶ Parse et nettoie les enregistrements avec **Spark**
- ▶ Écrit du Parquet structuré vers Storage 1
- ▶ **Sortie** : jeu de données propre

i MS2 – Scoring

- ▶ Lit les données propres depuis Storage 1
- ▶ Applique du **feature engineering**
- ▶ Calcule des scores dérivés avec Spark
- ▶ **Sortie** : jeu de données enrichi dans Storage 2

i MS3 – Prédiction

- ▶ Lit les données enrichies depuis Storage 2
- ▶ Charge le modèle Python exporté
- ▶ Exécute les prédictions
- ▶ **Sortie** : résultats exposés via Streamlit

⚠ Script d'entraînement Python

Script Python autonome qui lit Storage 2, entraîne un modèle ML (scikit-learn, MLlib, ...), et le **séréalise** (ex. onnx / pickle) pour que MS3 puisse le charger à l'exécution.

Livrables et évaluation

Livrables attendus

- 1 **Dépôt GitHub** – tout le code source Scala et Python.
- 2 Trois **microservices** fonctionnels communiquant via le stockage partagé.
- 3 Un **modèle ML Python** entraîné et exporté.
- 4 Un **dashboard Streamlit** affichant les prédictions par lots.
- 5 Une **présentation** (≤ 10 pages) : choix d'architecture, difficultés, résultats.

Notation (60% de la note finale)

Qualité de l'architecture	20%
Qualité du code Scala FP	30%
Pipeline de bout en bout	30%
Dashboard Streamlit	10%
Présentation	10%

Rappel

40% de la note finale provient des travaux pratiques en session.